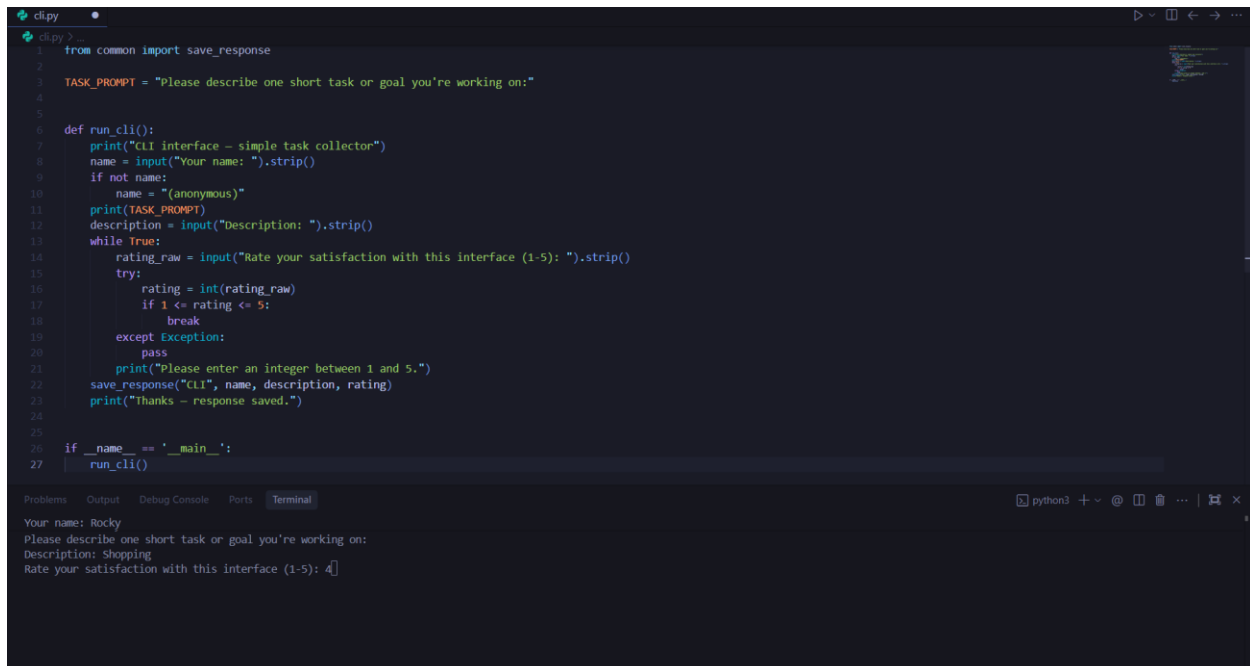




```
1 from common import save_response
2
3 TASK_PROMPT = "Please describe one short task or goal you're working on:"
4
5
6 def run_cli():
7     print("CLI interface - simple task collector")
8     name = input("Your name: ").strip()
9     if not name:
10         name = "(anonymous)"
11     print(TASK_PROMPT)
12     description = input("Description: ").strip()
13     while True:
14         rating_raw = input("Rate your satisfaction with this interface (1-5): ").strip()
15         try:
16             rating = int(rating_raw)
17             if 1 <= rating <= 5:
18                 break
19         except Exception:
20             pass
21         print("Please enter an integer between 1 and 5.")
22     save_response("CLI", name, description, rating)
23     print("Thanks - response saved.")
24
25
26 if __name__ == '__main__':
27     run_cli()
```

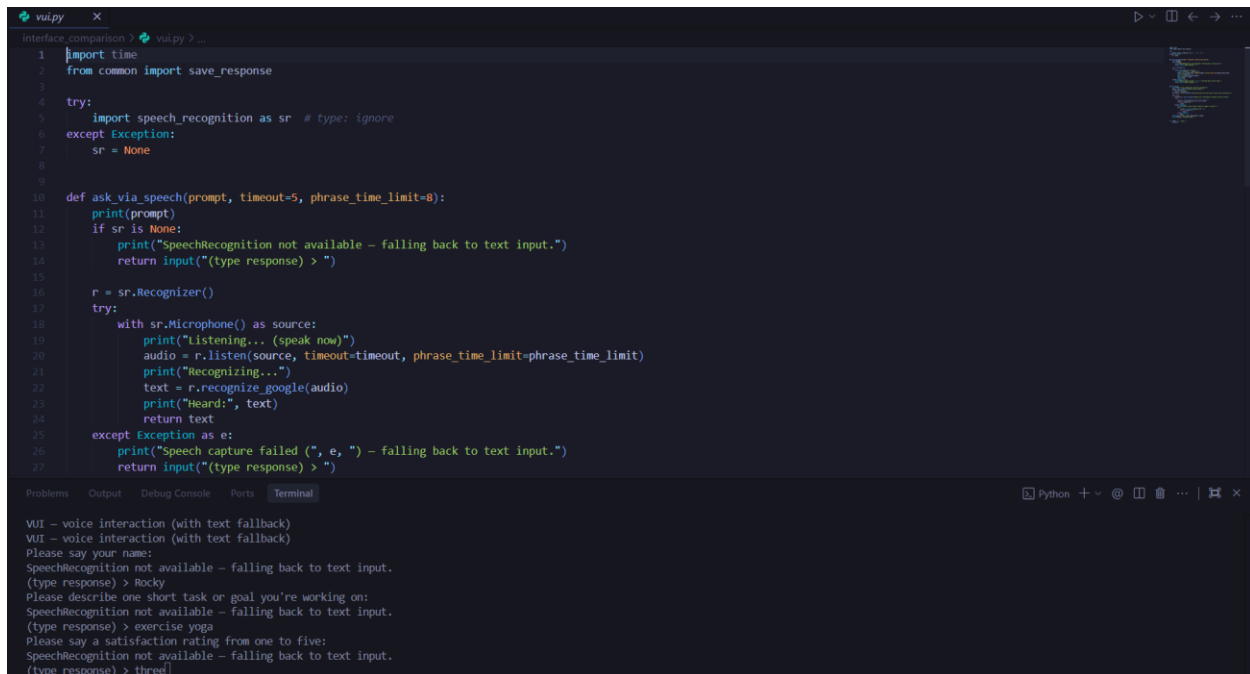
Problems Output Debug Console Ports Terminal

python3 + ~ @ ... | | x

Your name: Rocky
Please describe one short task or goal you're working on:
Description: Shopping
Rate your satisfaction with this interface (1-5): 4]

1. Command Line Interface (CLI)

- **Theory:** A text-based interface enables users to communicate with a computer using the command line, as opposed to using a graphical interface. CLI is generally faster and uses very little system resources compared to GUI applications.
- **Implementation:** Developed a Python script (`cli.py`) that utilizes standard `input()` functions to prompt the user sequentially for their name, the task they are working on, and a satisfaction rating from 1 to 5.
- **Output / Observation:** The interface successfully captured the username ("Rocky"), the task description ("Shopping"), and an integer satisfaction rating ("4").
 - The interaction was straightforward and text-heavy, requiring exact keyboard input to proceed.



```
1 import time
2 from common import save_response
3
4 try:
5     import speech_recognition as sr # type: ignore
6 except Exception:
7     sr = None
8
9
10 def ask_via_speech(prompt, timeout=5, phrase_time_limit=8):
11     print(prompt)
12     if sr is None:
13         print("SpeechRecognition not available - falling back to text input.")
14         return input("(type response) > ")
15
16     r = sr.Recognizer()
17     try:
18         with sr.Microphone() as source:
19             print("listening... (speak now)")
20             audio = r.listen(source, timeout=timeout, phrase_time_limit=phrase_time_limit)
21             print("Recognizing...")
22             text = r.recognize_google(audio)
23             print("Heard:", text)
24             return text
25     except Exception as e:
26         print("Speech capture failed (" + e + ") - falling back to text input.")
27         return input("(type response) > ")

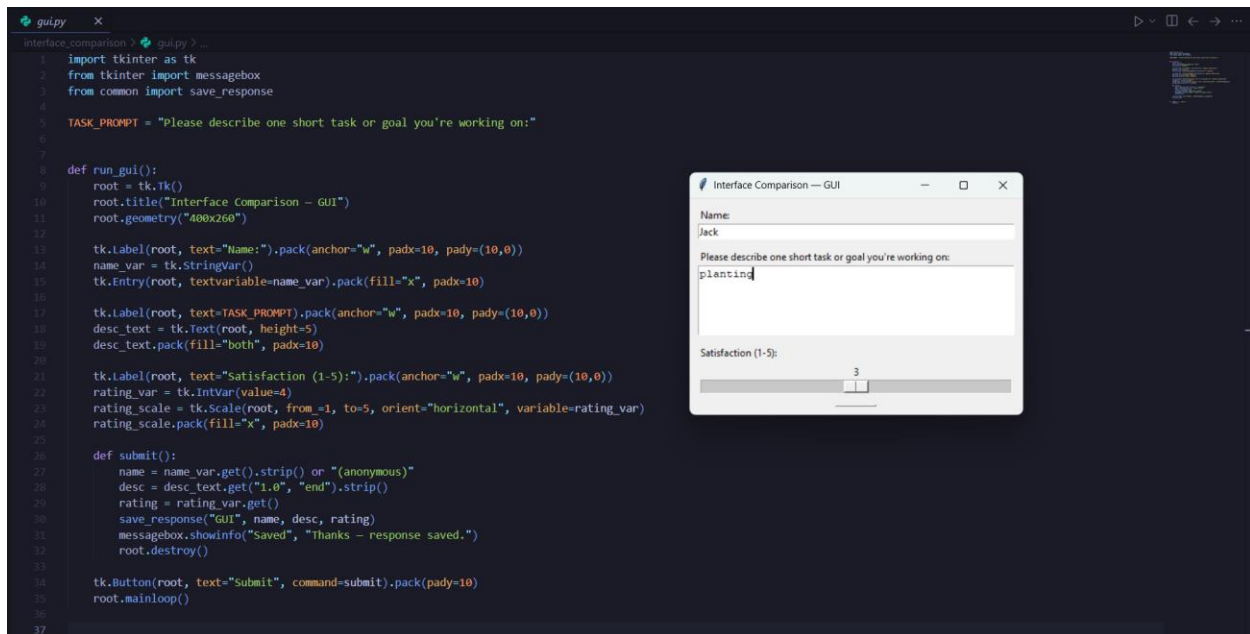
```

Problems Output Debug Console Ports Terminal

VUI - voice interaction (with text fallback)
VUI - voice interaction (with text fallback)
Please say your name:
SpeechRecognition not available - falling back to text input.
(type response) > rocky
Please describe one short task or goal you're working on:
SpeechRecognition not available - falling back to text input.
(type response) > exercise yoga
Please say a satisfaction rating from one to five:
SpeechRecognition not available - falling back to text input.
(type response) > three]

2. Graphical User Interface (GUI)

- **Theory:** A GUI uses visual elements including icons, buttons, and windows which makes it more user-friendly for users to work with the system. It is visually intuitive but more resource-intensive.
- **Implementation:** Used the tkinter library to build a visual window (`gui.py`). The layout features text labels, entry fields for the name and task description, a horizontal sliding scale (`tk.Scale`) for the satisfaction rating, and a "Submit" button.
- **Output / Observation:** * A clean, 400x260 pixel window titled "Interface Comparison - GUI" was generated.
 - The user ("Jack") typed "planting" into a multi-line text box and easily dragged the slider to "3" before submission. The visual layout made the process highly intuitive.



3. Voice User Interface (VUI)

- **Theory:** VUI allows natural and fluid communication by processing user speech, eliminating the need for typing or clicking. However, it faces accuracy challenges with background noise and complex commands.
- **Implementation:** Utilized the `speech_recognition` library (`vui.py`) to capture audio via the microphone. The script includes robust error handling with a graceful fallback to text input (`input()`) if the speech recognition module fails or cannot interpret the audio.
- **Output / Observation:** * During execution, the SpeechRecognition service was unavailable, triggering the except block.
 - The script successfully fell back to a terminal prompt, allowing the user ("Rocky") to manually type their task ("exercise yoga") and rating ("three"). This highlights a crucial design principle in VUI: always provide a secondary input method for hardware or environmental failures.

USER SATISFACTION COMPARISON:

To assess the interfaces, we evaluate them across the following parameters:

- **Efficiency:** CLI is the most efficient for advanced users who type quickly, while GUI offers moderate efficiency based on visual navigation. VUI efficiency is heavily dependent on speech recognition accuracy.
- **Learning Curve:** The GUI is easy to learn for most users. VUI has a very shallow curve since it mimics natural language, whereas CLI is steep for non-technical users.
- **Error Handling:** CLI is prone to typing errors and provides minimal feedback. GUI prevents errors by restricting inputs (e.g., using a slider for numbers 1-5 instead of a text box). VUI can misunderstand commands easily.

FINAL THOUGHTS:

The GUI is ideal for most users due to its intuitive nature and ease of use, preventing data entry errors through constrained inputs like sliders. The CLI is best for power users needing speed, and the VUI provides a convenient hands-free experience, provided the underlying recognition technology is functioning accurately.